

Handoff #5: the frontier ceiling is in, and Jev now leads the cascade

Everything since handoff #4 (commit 518ea11): GPT-5.6 replayed as the ceiling decider on all 1,076 airline states, GPT-OSS-20B finished, the cascade rebuilt around Jev, and a few project-level decisions about scope.

TL;DR

- U3 done: GPT-5.6 on airline arm 0 (5,380 calls, 0.1% failures, \$20.57). Strict 0.885 vs Jev 0.814: GPT-5.6 leads by +7.1 pts [+5.2, +9.1]. But Jev is 22× cheaper and 22× faster ($1.7\text{e-}4$ vs $3.8\text{e-}3$ \$/decision; p50 0.18 s vs 3.9 s), and better calibrated (ECE 0.029 vs 0.103). The labels are GPT-5.6's own actions, so its 0.885 is an upper bound.
- GPT-OSS-20B finished (all 5,380): strict 0.670, 10.9% failures (replies without the forced tool call). Jev - GPT-OSS = +15.5 pts, significant.
- The cascade's fast stage is now Jev, not Kev. Kev only exists to ask whether an open model gets near Jev, so it never stands in for it. Simulated Jev → GPT-5.6 at $t = 0.65$ (picked on held-out tasks): 0.854 held-out, at about a quarter of GPT-5.6's cost.
- Scope decisions: airline is the only headline domain and runs at full planned scope; mock gets no new runs (it ranks GPT-OSS above Jev, the opposite of airline). Frontier model = GPT-5.6, pinned as gpt-5.6-sol.
- Your job on Gilbreth: finish the Kev-4B cascade comparison row (4,005 calls, ~\$1.30). Command on page 5.

Your first 5 minutes

```
git fetch && git checkout jev-cascade-gpt56-pilot # this round is on a branch, not main yet
less results/replay/airline/report/summary.md # arm 0, now with GPT-5.6
less DECISIONS.md # new rows dated 2026-10-01/02
less PLAN.md # U3 now has its follow-up steps
```

1 · Where we are

Experimental arms (agent control, tau2 airline)

Arm	What	Status
0	each decider -> none (decision quality + cost)	PART — 8 deciders complete incl. GPT-5.6; Kev-4B cascade 4,005 calls left (Gilbreth); live Jev cascade not run yet
A	Jev (API) -> G	TODO — frontier G = GPT-5.6; pilot first to price it
B	G choice call -> G	PART — done with local G (0.308); frontier-G run TODO
C	G, one call (production default)	PART — done with local G (0.196); frontier-G run TODO
D	small LLM -> G	TODO
E	Kev (local GPU) -> G	PART — done with local G (0.048); frontier-G run TODO

Arm 0 is nearly finished. The in-loop arms with GPT-5.6 as G and as the user simulator are the paper's headline and the biggest remaining cost.

PLAN.md checklist

- Done: G1-G4, U0, U1, U2, and now U3 (Jev, GPT-OSS-20B and GPT-5.6 on all 1,076 airline states).
- Partial: U4 (harness built; B/C/E run with local G; every frontier-G run and arms A/D still to do).
- Not started: U5 (paper tables and figures), router and cache-guard decision points.

Decisions this round (rows in DECISIONS.md)

- Jev is the representative classifier. Wherever one classifier stands for "the classifier" (the cascade's fast stage, a headline row), it's Jev. Kev appears only next to Jev, as the open-weights comparison. A Kev-only test answers nothing the paper asks.
- Airline at full scope, no cost cuts (no cheaper user simulator, no fewer arms or trials). Mock is a pilot/appendix point only.
- Frontier = openrouter/openai/gpt-5.6-sol. OpenRouter's openai/gpt-5.6 is now an alias for it; other GPT-5.6 variants cost 0.1-2× as much, so it's pinned explicitly.
- Cascade thresholds come from a sweep, not 0.9. 0.9 was the vendor's starting advice for Jev, reused unchanged for Kev.

2 · Airline arm 0 with the ceiling

1,076 states from reward=1 episodes, 5 repeats each. Each (state, repeat) counts once, as its first attempt that wasn't an infra error.

Decider	Strict [95% CI]	Lenient	p50 latency	\$ / decision	ECE	Failures
GPT-5.6-sol (API, ceiling) †	0.885 [0.868, 0.901]	0.897	3.909 s	3.83e-3	0.103	0.1%
Jev (API)	0.814 [0.791, 0.836]	0.826	0.176 s	1.73e-4	0.029	0.0%
GPT-OSS-20B (API)	0.670 [0.646, 0.693]	0.686	1.299 s	3.11e-4	0.286	10.9%
Cascade Kev-4B -> GPT-OSS, †=0.9 *	0.669 [0.634, 0.704]	0.685	2.100 s	3.28e-4	0.287	7.2%
Kev-9B (A100)	0.401 [0.371, 0.431]	0.546	0.548 s	local	—	3.2%
Qwen3-8B (A100, vLLM)	0.396 [0.368, 0.425]	0.442	0.656 s	local	0.60	0.0%
Kev-4B (A100)	0.321 [0.294, 0.350]	0.445	0.439 s	local	0.124	3.2%
Kev-0.8B (A100)	0.273 [0.247, 0.300]	0.418	0.218 s	local	0.088	3.3%

† Graded against its own actions: the airline labels are GPT-5.6's reference trajectories, so 0.885 is an upper bound. * Incomplete: 4,005 calls still missing (729 of 1,076 states seen); finishing it is your Gilbreth job. Local deciders' latency and energy are unchanged from handoff #4.

Paired significance (cluster bootstrap over states)

Comparison (airline)	States	Diff	95% CI	Significant?
GPT-5.6 vs Jev	1,076	+7.1 pts	[+5.2, +9.1]	Yes
Jev vs GPT-OSS-20B	1,075	+15.5 pts	[+13.3, +17.7]	Yes
Jev vs Kev-9B	1,042	+40.7 pts	[+37.4, +44.1]	Yes

Reading it: the frontier model buys about 7 points over Jev, for 22× the cost and 22× the latency per decision. Jev is also the best calibrated decider here and the most consistent across repeats (0.985 vs GPT-5.6's 0.958). That trade-off is exactly what a cascade can exploit: let Jev decide when it's confident, and pay for GPT-5.6 only when it isn't.

3 · The cascade, rebuilt around Jev

`replay_decisions.py --deciders cascade now takes --cascade-fast {jev, kev}, default jev. Runs are named cascade-<fast>-t<threshold> (e.g. cascade-jev-t0.65), and a Jev cascade needs no Kev server.`

`kev_cascade_replay.s` `slurm` passes `--cascade-fast kev`, so it stays the Kev comparison row.

Why not Jev -> GPT-OSS-20B?

Escalating only helps toward a model that's stronger than the fast stage. On airline, GPT-OSS-20B is weaker than Jev (0.670 vs 0.814), so every escalation swaps a decision for a worse, costlier one. Simulated: 0.802 at $t=0.6$, 0.707 at $t=0.9$. The cascade escalates to GPT-5.6 instead.

Jev -> GPT-5.6, simulated from the two standalone runs

Threshold	Sent to GPT-5.6	Accuracy	\$ / decision	Mean latency	Live run $\approx$
0.6	13%	0.842	6.7e-4	0.87 s	\$3.60
0.65 (chosen)	17%	0.850 (0.854 held-out)	8.4e-4	1.09 s	\$4.50
0.7	21%	0.854	9.7e-4	1.29 s	\$5.20
0.8	31%	0.870	1.4e-3	1.81 s	\$7.30
0.9	52%	0.881	2.2e-3	2.84 s	\$11.60
GPT-5.6 alone	100%	0.885	3.8e-3	4.90 s	—

In-sample rows (threshold scored on the same states it is judged on); 0.65 also has its held-out score. How 0.65 was chosen: 40 random half/half splits by task; on one half, pick the Pareto point closest to utopia (the same rule as `make_cascade_figure.py`), score it on the other half. 0.65 won 23 of 40 folds (0.75 won 15) and is also the full-data pick. Held-out accuracy 0.854 (range 0.822–0.876).

Mock would have told the wrong story. On mock, GPT-OSS beats Jev (0.855 vs 0.772) and the Kev-4B cascade looks viable (0.680 at $t=0.9$, sometimes not escalating). On airline the ranking flips and the Kev-4B cascade escalates ~98% of calls, making it GPT-OSS plus Kev's cost. Worth one appendix paragraph: easy benchmarks flatter weak designs.

4 · What's next

Your job on Gilbreth: finish the Kev-4B cascade (comparison row)

Needs the Kev-4B server on an A100, so it can't run on the laptop. It resumes past the 3,169 calls already done. Escalations bill the OpenRouter key in Gilbreth's .env: ~\$1.30.

```
git fetch && git checkout jev-cascade-gpt56-pilot
sbatch --mem=32G
--export=ALL, STATES=results/states/airline.jsonl, RUN=results/replay/airline, THRESHOLD=0.9 \
  scripts/kev_cascade_replay.slurm
```

Then commit and push only calls-cascade-kev-4b-t0.9.jsonl and meta-cascade-kev-4b-t0.9.json. Nobody else writes those files, so there's nothing to merge.

Next paid steps (laptop, OpenRouter balance ~\$29)

- Live Jev → GPT-5.6 cascade at $t=0.65$, ~\$4.50: `--deciders cascade --cascade-threshold 0.65 --llm-model openrouter/openai/gpt-5.6-sol --provider-sort price`.
- U4 with a frontier G: arms A-E, GPT-5.6 as G and as the user simulator, 50 tasks × 5 trials. Pilot a few episodes first to measure \$/episode; this will need a new top-up.
- U5: paper tables and figures. The figure scripts' label tables don't know GPT-5.6 or the Jev cascade yet.

Gotchas we hit this round

- \$/call moved 2× during the run. The first 500 GPT-5.6 calls averaged \$0.0083 (cache cold), the whole run \$0.0038. Check the real \$/call after a few hundred calls rather than trusting a pilot of 25.
- The spending guard wants 1.25× the estimate in the account. When that doesn't fit, split the states file and run the halves one after another with half the cap each, rather than weakening the guard.
- A subset run used to shrink the run's `n_states` (a 5-state pilot set `airline` to 5). Fixed: it now only grows.
- Locally, tests need `TAU2_DATA_DIR` pointing at the tau2 data (e.g. `~/tau2-bench/data`); without it two label tests fail.

Changed files this handoff

```
scripts/replay_decisions.py # --cascade-fast, n_states only grows
scripts/kev_cascade_replay.slurm # passes --cascade-fast kev
results/replay/airline/calls-llm_openrouter_openai_gpt-5.6-sol.jsonl # U3
results/replay/airline/calls-llm_openrouter_openai_gpt-oss-20b.jsonl # completed
results/replay/airline/report/ # regenerated
DECISIONS.md, PLAN.md, scripts/make_handoff5_pdf.py
```